i want to build a proposal assistant which we can upload rfi or rfp after the upload we can see excactly the document inside the box as it is how it is ,the box top right corner have a add answer button when clicked on the (it will notify to select the question then once the question is selected then the press go button then rag pipeline runs answer is generated fotr the question and automatically get filled where the cursor was present when add answer was pressed (for eg if it is a excel file then answer is aaded to that particular cell if it is word then aswer is added to that particular likewiese for ppt) use most modern UI/UX libraray to make the frontend and like expert suggest me if it is possible and in the end when all thisng are done the user should be able to downliad the answer filled rfi

Short answer: yes, it's possible

The RAG part is the easy part. The hard part is **faithful rendering plus writing back to the exact cursor position in .xlsx, .docx and .pptx without breaking formatting**. That decision drives your whole architecture.

Two ways to build the document box

Option A: Embed a real office editor (my recommendation for the MVP)

Use **ONLYOFFICE Docs** (AGPL) or **Collabora Online** (MPL) inside an iframe or component.

- Gives true-fidelity rendering of Word, Excel and PowerPoint in the browser.
- Their plugin and Automation APIs let you read the current selection and cursor, and insert text at the cursor (a cell in Excel, a paragraph in Word, a text box in PPT).
- Saving and downloading the filled file is built in, so formatting is preserved.
- Trade-off: it's heavier to host, the UI is theirs (you wrap it with your own chrome), and the licence needs checking for commercial use.

Commercial alternatives with nicer SDKs: **Apryse WebViewer** and **Nutrient (PSPDFKit) Web SDK**.

Option B: Custom viewer plus server-side patching

- Preview: `docx-preview` (Word), Univer or Fortune-Sheet (Excel), `pptx-preview` (PPT), `pdf.js` (PDF).
- Each click maps to an anchor: `{sheet, cell}`, `{paragraph_index}`, or

`{slide, shape_id}`.

- The backend patches the original file with `openpyxl`, `python-docx` or `python-pptx`, then the preview refreshes.
- Trade-off: you get full UI control, but PPT and Word preview fidelity is mediocre, and mapping browser clicks to file anchors is fiddly. It's best as a later phase if you need to own the whole experience.

Don't forget PDFs. Many RFPs arrive as PDF. You can't truly edit them, so you'd overlay text with `pdf-lib`, or export answers to a separate response document.

The flow you described

- User clicks **Add Answer**. The app records the answer anchor (the cursor location) and enters "select question" mode.
- User highlights the question text. A toast confirms what was captured, and the **Go** button activates.
- The backend receives the question, its context (section heading, Excel column headers, neighbouring rows), and the document type.
- The RAG pipeline runs and streams the answer.
- The answer is inserted at the stored anchor, highlighted as "AI draft" so the user can review, edit or undo.
- **Download** exports the original file with the answers filled in.

UX tip: for Excel, auto-detect the likely answer cell (the empty cell to the right of the question) and let the user override it. That saves a lot of clicks.

RAG pipeline

- **Knowledge base:** past proposals, product docs, security and compliance docs, case studies.
- **Parsing:** Docling or Unstructured, with structure-aware chunking by headings and tables.
- **Retrieval:** hybrid search (BM25 plus vector) in Postgres with pgvector or Qdrant, then a reranker (bge-reranker or Cohere Rerank).
- **Generation:** Claude with the retrieved chunks, a company tone prompt, and word or character limits (Excel cells cap at 32,767 characters).
- **Trust features that matter for RFPs:** source citations, a confidence score, and "no supporting source found" instead of hallucinating. A wrong compliance answer in an RFP is costly.

Suggested stack

Layer	Choice
Frontend	Next.js, Tailwind, shadcn/ui (Radix), Framer Motion, Zustand, TanStack Query
Streaming	Vercel AI SDK or SSE
Doc engine	ONLYOFFICE or Collabora (MVP)
Backend	FastAPI, Celery with Redis
Data	Postgres with pgvector, S3 or MinIO for files
LLM	Claude API

Pitfalls to plan for

- **Excel:** merged cells, data-validation dropdowns, locked cells and macros (.xlsm).
- **Word:** questions inside tables, content controls, tracked changes.
- **PPT:** text boxes versus placeholders, and overflow when long answers don't fit a shape.
- **Style inheritance:** the inserted text should match the surrounding font and format.
- **Security:** RFPs are confidential, so offer a private deployment or at least encrypted storage, tenant isolation, and a no-training guarantee.
- **Licensing:** check ONLYOFFICE's AGPL terms and connection limits before commercial launch.

Suggested phases

- **MVP:** DOCX and XLSX only, ONLYOFFICE embedded, single-question flow with a basic RAG.
- **V2:** PPTX and PDF, citations, answer review states, and a knowledge-base manager.
- **V3:** "Auto-detect all questions and fill everything" bulk mode, team collaboration, and an answer library that learns from approved answers.

I can sketch the ONLYOFFICE plugin code for capturing the selection and inserting the answer, or draft the FastAPI and RAG service structure. Which would you like first?

you donot worry about rag pipeline and all just the ui of what i am thining off

is very dynamic and answer filling in that step is possible or not

Yes, the UI and the fill-at-cursor step are both possible

What you describe is a **guided "pick target, pick question, fill" flow** on top of a document viewer. The only real question is how exactly you can capture the cursor and write back, and that differs by file type and viewer.

Feasibility by file type

Format	Capture cursor	Write answer	Difficulty
Excel	Exact cell (`Sheet1!C14`)	Set cell value	Easy
Word	Paragraph or table cell	Insert text there	Medium
PPT	Slide plus text box	Fill that shape	Medium-hard
PDF	x/y position on a page	Overlay text only	Hard (no true editing)

How the box works, two ways

1. Embedded editor (ONLYOFFICE or Collabora)

- The document looks exactly like the original because it is the real editor.
- Your "Add Answer" button sits in your own React UI, floating top-right over the box.
- The editor runs in an iframe, so your app talks to it through its connector or plugin API. It asks "where is the cursor or selection?" and later says "insert this text here."
- Excel, Word and PPT all work the same way from your side, which makes this the **only approach that gives true "exactly as it is" fidelity for all three**.

2. Custom viewer plus a backend that edits the file

- Excel: Univer gives a near-perfect view, and a click gives the exact cell.
- Word: `docx-preview` renders well, but you must tag each paragraph with an index so a click maps to the file.
- PPT: browser renderers are weak. The better trick is to convert slides to images with LibreOffice and overlay invisible clickable regions for each shape.
- The backend writes with openpyxl, python-docx and python-pptx, then

re-renders.
- This gives full UI freedom, but "exactly as it is" is not guaranteed for Word or PPT.

The UI state machine

```
IDLE → click "Add Answer"
  → PICK_TARGET   (banner: "Click where the answer should go")
  → PICK_QUESTION (banner: "Now select the question", Go
disabled)
  → READY         (question chip shown, Go enabled)
  → GENERATING    (streaming, shimmer at target location)
  → REVIEW        (answer inserted, highlighted, Accept /
Regenerate / Undo)
  → IDLE
```

Two details matter here:

- You can skip the separate target step if the user's cursor is already placed when they click Add Answer, which matches what you described. Store the cursor as an anchor the moment the button is pressed, because clicking the button may make the document lose focus.
- Highlight the filled answer as an "AI draft" until the user accepts it, so they can see and undo what changed.

Things that will bite you

- **Focus loss:** clicking your external button can clear the selection in the editor. Save the anchor on `mousedown`, before focus moves.
- **Excel merged cells:** the answer must go in the top-left cell of the merge.
- **Word tables:** a question in one cell with the answer in the next cell is common, so offer "answer goes in the adjacent cell" as a smart default.
- **PPT overflow:** long answers will spill out of a text box, so you need auto-shrink or a warning.
- **Formatting:** inherit the font and style of the target spot rather than inserting plain text.
- **Download:** with the embedded editor this is trivial and the original formatting is kept. With the custom approach you download the file your backend patched.

My recommendation

If "see the document exactly as it is" and "fill at the cursor" are your top priorities, use **ONLYOFFICE embedded in Next.js, wrapped with your own shadcn/ui chrome**. Build the state machine above in Zustand, with Framer Motion for the banners and toasts. That gets you a working demo for Excel and Word fastest. Do PPT and PDF afterwards.

Want me to build a clickable UI prototype of this flow, with a mock document box, the Add Answer button, the step banners and the fill animation, so you can see how it feels before wiring up the real editor?

lets start it with excel crete brd ,prd and frd

BRD – Proposal Assistant (Excel MVP)
DOC · Open in Claude

PRD – Proposal Assistant (Excel MVP)
DOC · Open in Claude

FRD – Proposal Assistant (Excel MVP)
DOC · Open in Claude